

```

In [2]: import re
import pandas as pd
from collections import Counter
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer, ENGLISH_STOP_WORDS
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

FILE = r"D:\Datascience-MachineLearning\Datasets\IMDB_Dataset.csv"
TEXT_COL, LABEL_COL = "review", "sentiment"

df = pd.read_csv(FILE)

def clean(text):
    text = re.sub(r"<.*?>", " ", text.lower())
    return re.sub(r"^[a-z\s]", "", text)

df["clean"] = df[TEXT_COL].apply(clean)

# ---- sentiment model ----
X_train, X_test, y_train, y_test = train_test_split(
    df["clean"], df[LABEL_COL], test_size=0.2, random_state=42)

vec = TfidfVectorizer(stop_words="english", max_features=5000)
model = LogisticRegression(max_iter=1000)
model.fit(vec.fit_transform(X_train), y_train)
acc = accuracy_score(y_test, model.predict(vec.transform(X_test)))

# ---- frequent words per sentiment ----
def top_words(sentiment, n=10):
    words = (w for r in df[df[LABEL_COL] == sentiment]["clean"] for w in r.split()
              if w not in ENGLISH_STOP_WORDS and len(w) > 2)
    return Counter(words).most_common(n)

pos, neg = top_words("positive"), top_words("negative")
table = pd.DataFrame({
    "Positive word": [w for w, _ in pos], "Count ": [c for _, c in pos],
    "Negative word": [w for w, _ in neg], "Count": [c for _, c in neg],
}, index=range(1, 11))

```

```

print("=" * 52)
print(f" IMDb Sentiment Analysis | Accuracy: {acc:.2%}")
print("=" * 52)
print("\n Top 10 most frequent words\n")
print(table.to_string())

```

```

=====
IMDb Sentiment Analysis | Accuracy: 88.46%
=====

```

Top 10 most frequent words

	Positive word	Count	Negative word	Count
1	film	40167	movie	48526
2	movie	36709	film	35868
3	like	17181	like	21885
4	good	14573	just	20903
5	just	13981	bad	14376
6	great	12812	good	14340
7	story	12613	really	12294
8	time	12130	time	11841
9	really	10720	dont	10283
10	films	8584	story	9908

In []: